Práctica guiada: visualizador de un UR3e por TCP/IP

Redes de Comunicación Industriales

Ficha de la práctica	
Objetivo	Programa en Python que lee el estado de un UR3e por TCP/IP y lo reproduce en un simulador PyBullet (visualizador)
Interfaces del robot	Primaria (puerto 30001) y tiempo real (puerto 30003); en el sobresaliente, envío de órdenes URScript
Entorno de trabajo	Desarrollo en el aula de prácticas (red 10.172.18.x) con URSim en una máquina virtual de VirtualBox en modo puente; validación final con los UR3e del laboratorio de robótica (red 10.172.21.x)
Código de partida	Solo el visualizador (paquete `UR3e_rg2`). Todo el código de comunicación lo escribís vosotros
Entrega	Código, memoria breve y defensa en el laboratorio
Calificación	Por niveles, incremental y binaria: aprobado (5), notable (7), sobresaliente (10)

Contenido

- **Contexto:** 1. Objetivos, prerrequisitos y material · 2. Aula y laboratorio de robótica · 3. Entorno: URSim en VirtualBox
- **Fundamentos técnicos:** 4. Capas · 5. Interfaces cliente · 6. Tramas de la interfaz primaria · 7. Interfaz de tiempo real · 8. Lectura del buffer · 9. El visualizador
- **Trabajo a realizar:** 10. Desarrollo guiado por hitos
- **Evaluación:** 11. Entrega y evaluación · 12. Preguntas de comprensión

1. Objetivos, prerrequisitos y material

Al terminar esta práctica habréis construido un **visualizador**: un programa en Python que se conecta por TCP/IP a un robot UR3e (real o simulado con URSim), decodifica byte a byte los mensajes binarios que el robot emite y reproduce su configuración articular en un simulador PyBullet, de modo que el robot de la pantalla se mueve igual que el robot real.

Todo el código de comunicación lo escribís vosotros. Solo se os proporciona el visualizador, un paquete de Python que abre la ventana de PyBullet y permite colocar y mover uno o varios robots UR3e.

Lo que vais a aprender

- Abrir y mantener una conexión TCP cliente con un dispositivo industrial y entender qué papel juega cada capa (Ethernet, IP, TCP, aplicación).
- Interpretar un protocolo binario propietario a partir de su documentación: longitudes, tipos, orden de bytes (*endianness*) y estructura anidada mensaje/subpaquete.
- Resolver el problema clásico de TCP como flujo de bytes: los mensajes no llegan "de uno en uno", hay que delimitarlos.
- Comparar dos interfaces del mismo equipo con filosofías distintas: la **primaria** (mensajes de longitud variable, 10 Hz) y la de **tiempo real** (estructura fija, alta frecuencia).
- Configurar una máquina virtual en red con modo puente y diagnosticar problemas de conectividad.
- En el nivel máximo: enviar órdenes al robot y cerrar el lazo comprobando, a partir del propio estado que publica, que las órdenes se han ejecutado.

Prerrequisitos

- Modelo por capas, direccionamiento IP, máscara de subred y puertos TCP (teoría de la asignatura).
- Python básico: funciones, clases, listas y diccionarios. Se usarán los módulos estándar `socket`, `struct` y `threading`.

Material

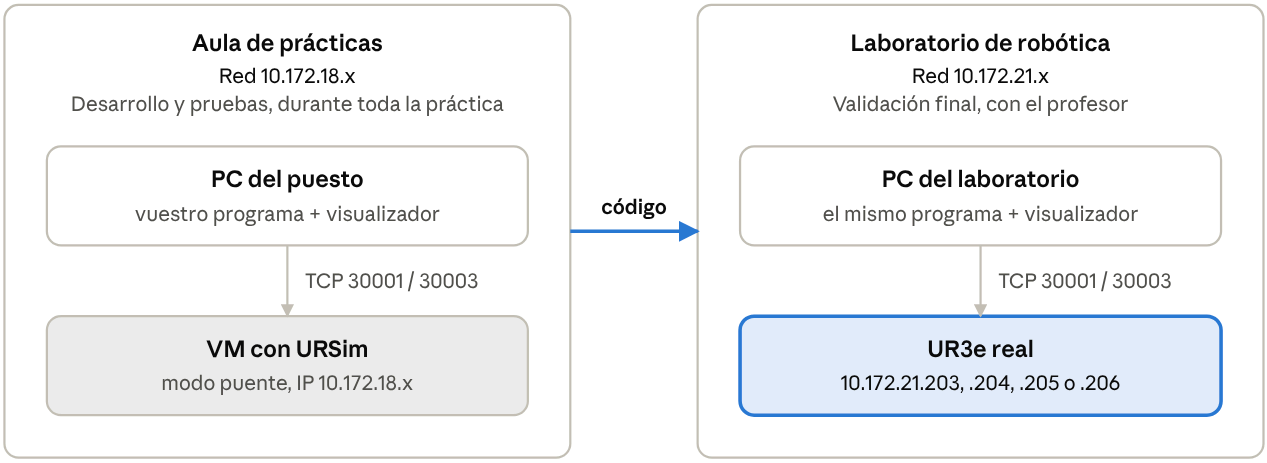
Elemento	Descripción
PC del puesto	Python 3 y el visualizador `ur3e_rg2-0.1.0-py3-none-any.whl` (instalación en el apartado 9)
VirtualBox	Máquina virtual con URSim (simulador oficial de Universal Robots, PolyScope 5)
Robots UR3e reales	Solo para la validación final en el laboratorio de robótica, siempre con el profesor presente
Documentación	*Primary/Secondary Client Interface* de Universal Robots (versiones de software 5.9 y 5.10) y la hoja de la interfaz de tiempo real (*Client Interfaces*), disponibles en el aula virtual

> **Seguridad.** Ningún programa se conecta a un robot real hasta que haya funcionado sin errores contra URSim. Cualquier orden de movimiento a un robot real requiere autorización expresa del profesor.

2. Dónde se trabaja: aula de prácticas y laboratorio de robótica

La práctica se desarrolla en el **aula de prácticas** (red 10.172.18.x) contra URSim. Cuando vuestro programa funciona, se valida con los robots reales en el **laboratorio de robótica** (red 10.172.21.x).

Se desarrolla en el aula contra URSim y se valida con los robots reales



Al pasar del aula al laboratorio solo cambia la IP de destino: de la VM al robot

Dos espacios y dos redes · desarrollo en el aula, validación en el laboratorio de robótica

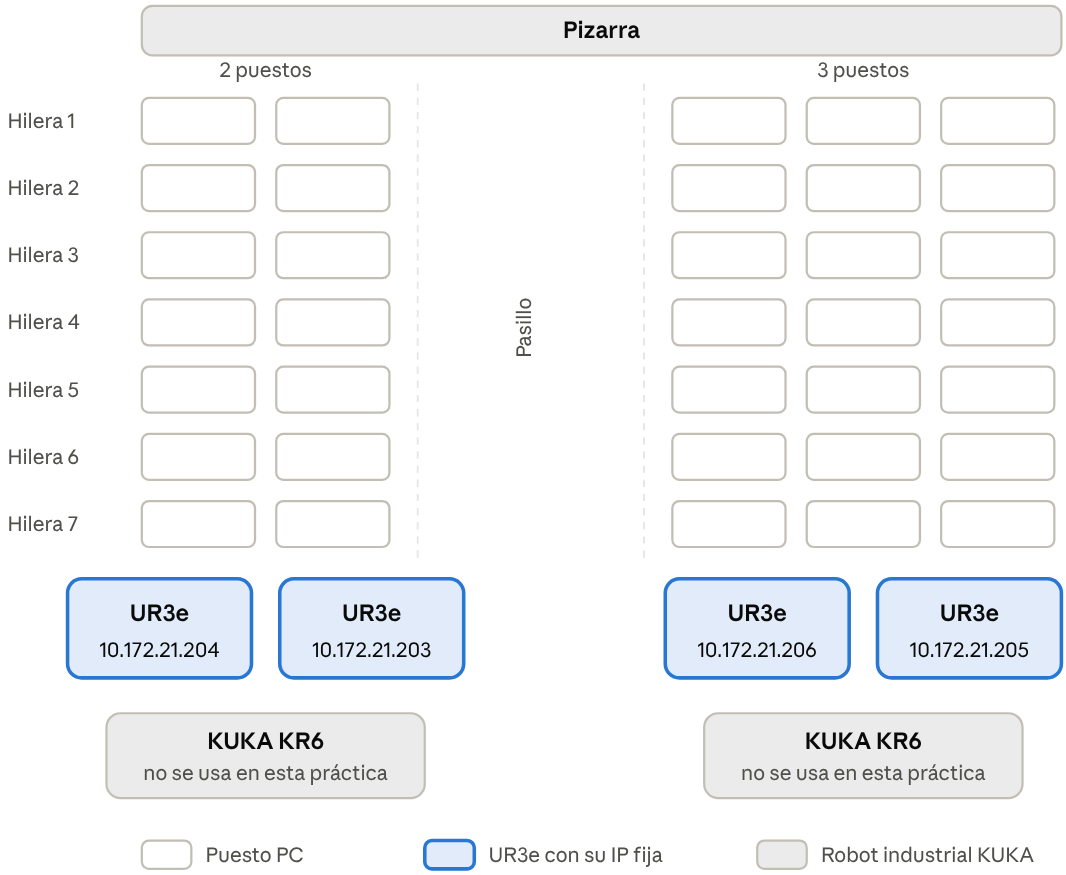
	Aula de prácticas	Laboratorio de robótica
Red	10.172.18.x	10.172.21.x
Robot	URSim en vuestra VM (o en la de otro puesto)	UR3e reales: .203, .204, .205 y .206
Máquina virtual	Sí, en modo puente	No hace falta: el PC se conecta directamente al robot
Para qué	Desarrollo, pruebas y depuración de todos los niveles	Validación final y defensa, con el profesor presente

Los robots reales solo se usan desde los PCs del laboratorio de robótica. No intentéis conectaros a ellos desde el aula.

Laboratorio de robótica: situación de los UR3e

Los cuatro UR3e del laboratorio de robótica tienen IP fija en la red 10.172.21.x: los dos de la columna izquierda son el **.204** y el **.203**, y los dos de la derecha el **.206** y el **.205**.

Cuatro UR3e accesibles por red, tras las 7 hileras de puestos



Laboratorio · 7 hileras de 5 puestos, 4 UR3e y 2 KUKA KR6

Validación en el laboratorio de robótica

La validación comprueba que el mismo programa que funciona contra URSim funciona con un robot real cambiando solo la IP de destino.

1. **Llevad vuestro código** y el fichero de configuración de robots. Si el visualizador no está instalado en el PC del laboratorio, instaladlo con los pasos del apartado 9.
2. **Comprobad la conexión con el robot** asignado: `ping` a su IP y prueba del puerto 30001, igual que hicisteis con la VM.
3. **Cambiad la IP de destino** en el fichero de configuración, sin tocar el código.
4. **Empezad solo leyendo**: el robot de la pantalla debe seguir al real mientras el profesor lo mueve desde su consola.
5. **Las órdenes de movimiento** (sobresaliente) solo se envían con autorización y presencia del profesor.

Anotad en la memoria qué robot real habéis usado y cualquier diferencia que hayáis observado respecto a URSim.

3. Entorno: URSim en VirtualBox con red en modo puente

URSim se ejecutará en una máquina virtual de VirtualBox configurada con el adaptador de red en **modo puente (*bridge*)**, para que vuestro programa hable con el robot simulado exactamente igual que con uno real.

URSim es el mismo software de control (PolyScope) que ejecuta el armario del robot, pero sin el brazo físico. Lo importante para nosotros es que abre **los mismos puertos TCP y envía los mismos mensajes** que un UR3e real. Si vuestro programa funciona contra URSim, funcionará contra el robot cambiando únicamente la IP de destino.

Qué hace el modo puente

VirtualBox ofrece varios modos para conectar la tarjeta de red virtual de la VM:

Modo	Qué ve la red del laboratorio	¿Se puede abrir una conexión desde fuera hacia la VM?
NAT (por defecto)	Solo al PC anfitrión. La VM tiene una IP privada interna (típicamente 10.0.2.15) que nadie más ve	No, salvo que se configure reenvío de puertos
Solo anfitrión (*host-only*)	Nada: es una red privada entre el PC y la VM	Solo desde el propio PC
Puente (*bridge*)	Un equipo más, con su propia MAC y su propia IP en la red del aula, 10.172.18.x	Sí, desde cualquier equipo de la red

En modo puente, VirtualBox conecta la tarjeta virtual de la VM directamente a la tarjeta física del PC, a nivel de enlace (capa 2). Las tramas Ethernet de la VM salen al switch con su propia dirección MAC. El switch aprende esa MAC en el puerto del PC como si hubiera dos equipos conectados al mismo cable, y la VM obtiene su propia dirección IP.



NAT frente a puente · dónde queda la VM en la red

Por qué usamos el modo puente

- **La interfaz del robot es un servidor.** El robot (o URSim) escucha en los puertos 30001, 30002 y 30003 y vuestro programa es el cliente que abre la conexión. En NAT la VM no acepta conexiones entrantes sin configurar reenvío de puertos; en puente no hace falta.
- **Mismo código para simulador y robot real.** URSim en modo puente se alcanza igual que un UR3e: con su IP y su puerto. Al pasar al laboratorio de robótica solo cambia la IP de destino.
- **Varios simuladores a la vez.** Para el nivel de notable necesitáis varios robots: podéis conectaros a la VM de vuestro compañero de puesto igual que a la vuestra.

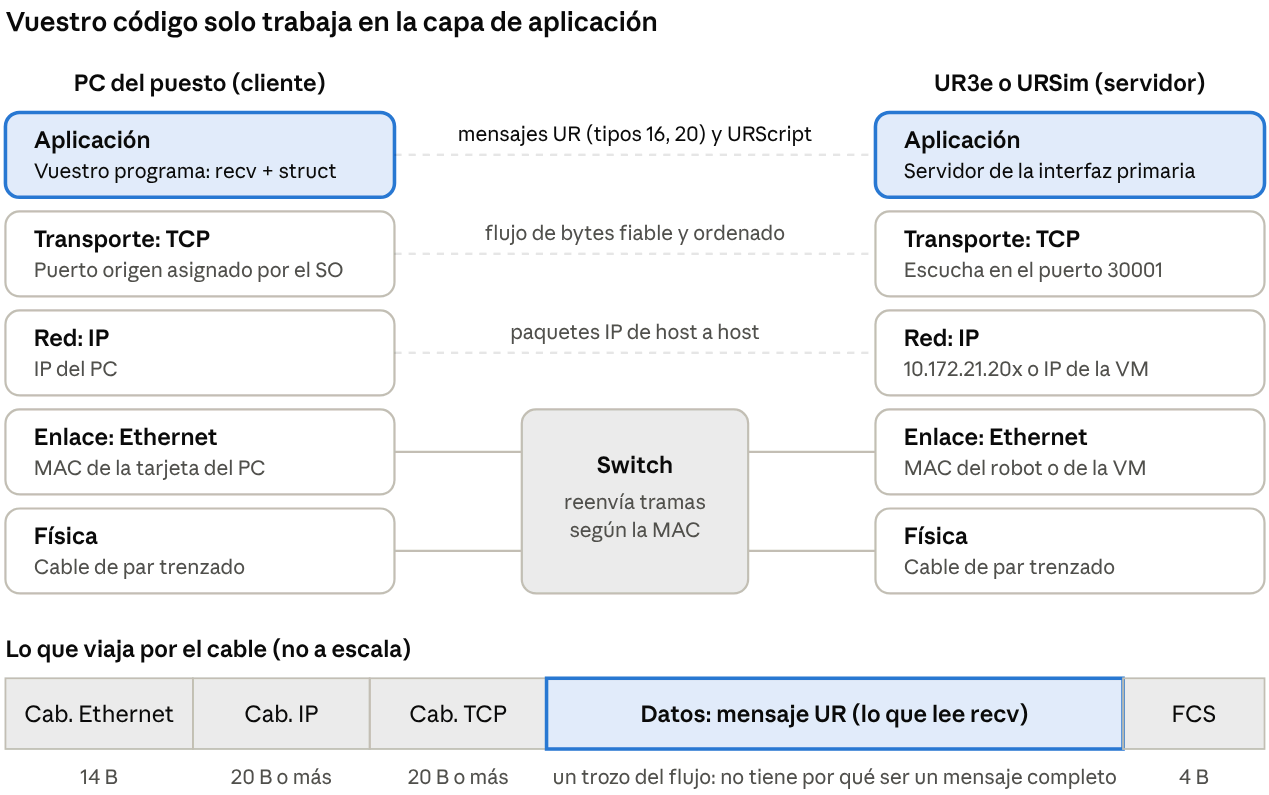
Pasos de configuración

1. Con la VM apagada: *Configuración → Red → Adaptador 1 → Conectado a: Adaptador puente*, y en *Nombre* elegid la tarjeta Ethernet cableada del PC (no una Wi-Fi ni una virtual).

2. Arrancad la VM y averiguad la IP que ha recibido, que debe estar en la red 10.172.18.x (en un terminal de la VM: `ip addr`).
3. Desde el PC, comprobad que llega: `ping <IP_de_la_VM>`.
4. Arrancad URSim, encended el robot simulado desde PolyScope (botón de inicio, *ON* y *START*) y dejadlo en estado *Normal*.

4. Capas que intervienen en la comunicación

Vuestro programa solo escribe código en la capa de aplicación: abre un *socket* TCP y recibe bytes. Todo lo que hay por debajo lo resuelve el sistema operativo, pero necesitáis entenderlo para diagnosticar fallos.



Pila de capas PC–robot · encapsulado de un mensaje UR

Capa	Qué hace en esta práctica	Quién se encarga	Síntoma típico si falla
Física	Transmite los bits por el cable hasta el switch	Tarjeta de red, cable	Sin luz de enlace en la tarjeta o el switch

Capa	Qué hace en esta práctica	Quién se encarga	Síntoma típico si falla
Enlace (Ethernet)	Entrega tramas dentro de la red local usando direcciones MAC. En modo puente la VM tiene su propia MAC	Tarjeta de red, switch, VirtualBox	La VM no obtiene IP; `ping` falla aunque la IP sea correcta
Red (IP)	Identifica a cada equipo (10.172.18.x en el aula, 10.172.21.x en el laboratorio de robótica) y encamina los paquetes	Sistema operativo	`ping` falla: red o máscara equivocadas, IP duplicada
Transporte (TCP)	Abre una conexión con el puerto 30001 y entrega un **flujo de bytes** fiable y ordenado: retransmite lo perdido y reordena lo desordenado	Sistema operativo	*Connection refused* (nadie escucha en ese puerto) o *timeout* (un cortafuegos o un equipo apagado)
Aplicación	Da significado a los bytes: tamaño, tipo, subpaquetes, números en *big-endian*	**Vosotros**	Los datos llegan pero los valores son absurdos: error de desplazamiento, de tipo o de orden de bytes

La idea clave de la práctica: TCP garantiza que los bytes llegan todos y en orden, pero **no conserva los límites de los mensajes**. Una llamada a `recv()` puede devolver medio mensaje, uno entero o un mensaje y medio. Delimitar los mensajes es responsabilidad de la capa de aplicación; por eso cada mensaje UR empieza con su longitud.

5. Las interfaces cliente del UR3e

El controlador del UR3e es un **servidor TCP** que escucha en varios puertos; cada puerto es una interfaz distinta con su propio formato. En esta práctica usáis la primaria (30001) y la de tiempo real (30003).

Puerto	Interfaz	Qué envía el robot	Frecuencia	¿Acepta órdenes?	Uso en la práctica
30001	Primaria	Estado del robot (mensaje de tipo 16) + mensajes del sistema: versión, errores, excepciones de programa, *ventanas emergentes*…	10 Hz	Sí, programas URScript en texto	Aprobado y sobresaliente
30002	Secundaria	Solo estado del robot y mensaje de versión	10 Hz	Sí, URScript	Opcional
30003	Tiempo real	Un paquete de **estructura fija** con posiciones, velocidades, corrientes, etc.	500 Hz en e-Series	Sí, URScript	Aprobado
30011 / 30012	Primaria / secundaria de solo lectura	Lo mismo que 30001 / 30002	10 Hz	No	Recomendada mientras solo leáis
30004	RTDE	Las variables que el cliente solicita, con protocolo de negociación	Configurable	Sí	No (se menciona como referencia)
29999	*Dashboard server*	Respuestas de texto a comandos como cargar o arrancar un programa	Bajo demanda	Sí, comandos de texto	Extensión opcional

Origen de los datos: puertos, frecuencia de 10 Hz y contenido de 30001, 30002, 30011 y 30012 según el documento *Primary/Secondary Client Interface* de Universal Robots. Los datos de 30003, 30004 y 29999 proceden de la documentación general de Universal Robots para e-Series; contrastadlos con la versión de PolyScope de vuestra VM.

Dos filosofías de diseño

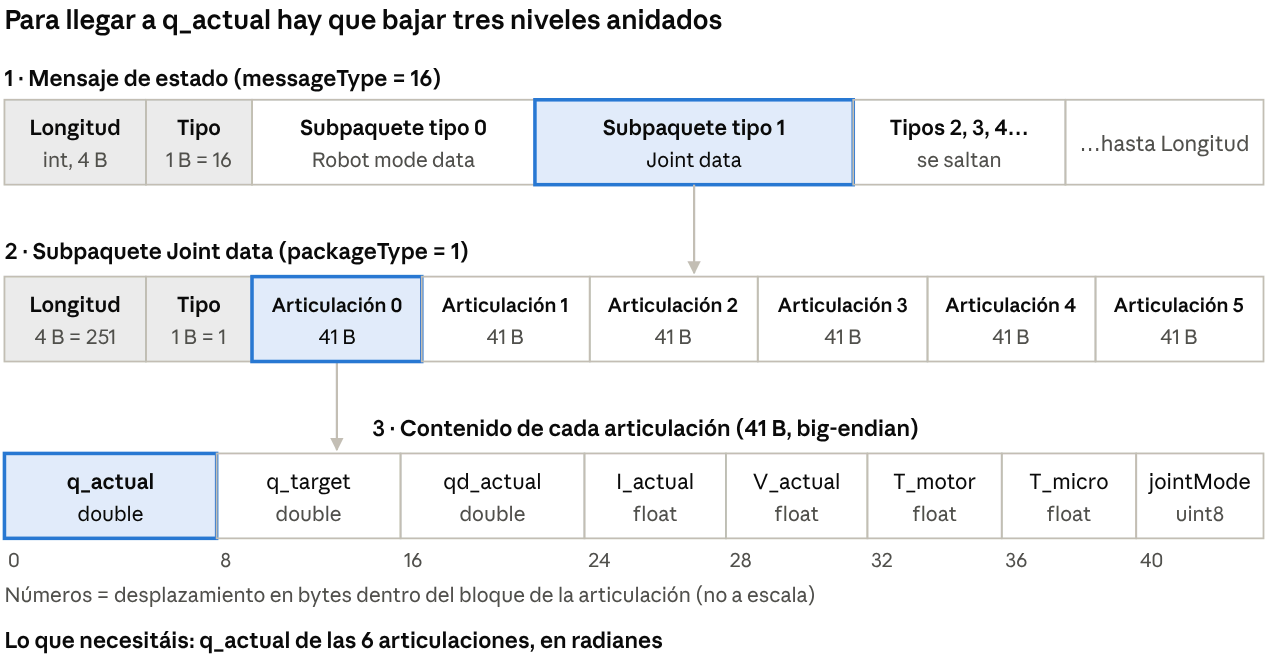
La primaria y la de tiempo real resuelven el mismo problema (contar al cliente qué hace el robot) de dos maneras opuestas. Entender por qué es uno de los objetivos de la práctica.

	Interfaz primaria (30001)	Interfaz de tiempo real (30003)
Estructura	Mensajes de longitud variable formados por subpaquetes, cada uno con su propia cabecera de longitud y tipo	Un único tipo de paquete con todos los campos siempre en el mismo sitio (posición fija en bytes)
Ventaja	Extensible: se pueden añadir subpaquetes nuevos sin romper a los clientes antiguos, que simplemente los saltan	Muy simple y rápida de decodificar: basta saber el desplazamiento (*offset*) del campo
Inconveniente	Hay que recorrer la estructura para encontrar lo que interesa	Cualquier cambio de versión que añada campos cambia la longitud del paquete; el cliente debe conocer la versión
Pensada para	Supervisión, interfaz de usuario, diagnóstico	Seguimiento rápido del movimiento

> **Antes de enviar órdenes (nivel máximo).** En PolyScope 5 los programas URScript que llegan por red pueden estar bloqueados según la configuración de *Control remoto* y de los servicios de red en los ajustes de seguridad. Si vuestro programa envía una orden y el robot no reacciona, revisad esos ajustes en URSim y leed los mensajes de error que la propia interfaz primaria os devuelve.

6. Las tramas de la interfaz primaria

El dato que necesitáis (el ángulo actual de cada articulación, `q_actual`) está tres niveles por debajo: dentro del **mensaje de estado** (tipo 16), en el **subpaquete de datos articulares** (tipo 1), repetido una vez **por cada una de las 6 articulaciones**. Todo lo que sigue procede del documento *Primary/Secondary Client Interface*, secciones 2 y 3.



Estructura anidada del mensaje de estado · mensaje → subpaquete → articulación

Regla de oro: cada bloque dice cuánto mide

Tanto el mensaje como cada subpaquete empiezan con una cabecera de 5 bytes: una longitud (`int` , 4 bytes) y un tipo (`unsigned char` , 1 byte). **La longitud incluye la propia cabecera.** Gracias a ello podéis saltar cualquier bloque que no os interese sin saber qué contiene: `siguiente = inicio + longitud` . Por eso el documento dice que los mensajes no documentados deben ignorarse.

Tipos de mensaje que vais a recibir

messageType	Mensaje	Cuándo llega	Qué hacer
20, con `robotMessageType` = 3	*Version message*	Una sola vez, nada más conectar; es el primer paquete	Leer la versión de software (útil para la interfaz de tiempo real)
16	*Robot State Message*	Periódicamente, 10 veces por segundo	Recorrer sus subpaquetes

messageType	Mensaje	Cuándo llega	Qué hacer
20, otros subtipos	*Robot Messages*: modo de seguridad (5), errores (6), excepción de programa (10), texto (0)…	Cuando ocurre un evento	Ignorar en el aprobado; mostrar en el sobresaliente
25	Variables globales del programa	Cuando hay un programa con variables	Ignorar
Cualquier otro	Uso interno	En cualquier momento	Saltar usando su longitud

Subpaquetes del mensaje de estado (tipo 16)

packageType	Nombre	Contenido útil	Nivel en que lo necesitáis
0	*Robot mode data*	`isProgramRunning`, `isProtectiveStopped`, `robotMode`, `speedScaling`	Sobresaliente (saber si el robot ha terminado un movimiento)
1	***Joint data***	`q_actual`, `q_target`, `qd_actual` … por cada articulación	**Aprobado**
2	*Tool data*	Entradas analógicas y tensión de la herramienta	No
3	*Masterboard data*	`digitalInputBits`, `digitalOutputBits` (enteros de 4 bytes, uno por bit)	No
4	*Cartesian info*	Posición X, Y, Z y orientación Rx, Ry, Rz de la herramienta	Opcional

packageType	Nombre	Contenido útil	Nivel en que lo necesitáis
5, 6	*Kinematics info*, *Configuration data*	Parámetros DH, límites	Solo se envían al inicializar o si cambia la configuración
7 a 13	Fuerza, información adicional, calibración, seguridad, herramienta, singularidades	Varios; algunos son de uso interno	No: saltarlos

Cómo convertir bytes en números

Todos los números viajan en **big-endian** (orden de red: el byte más significativo primero). En Python se decodifican con el módulo `struct`, empezando siempre el formato por `>`:

Tipo en el documento	Bytes	Formato `struct`
`int` / `int32_t`	4	`>i`
`uint32_t`	4	`>I`
`unsigned char` / `uint8_t` / `bool`	1	`>B` (o `>?` para bool)
`uint64_t`	8	`>Q`
`float`	4	`>f`
`double`	8	`>d`

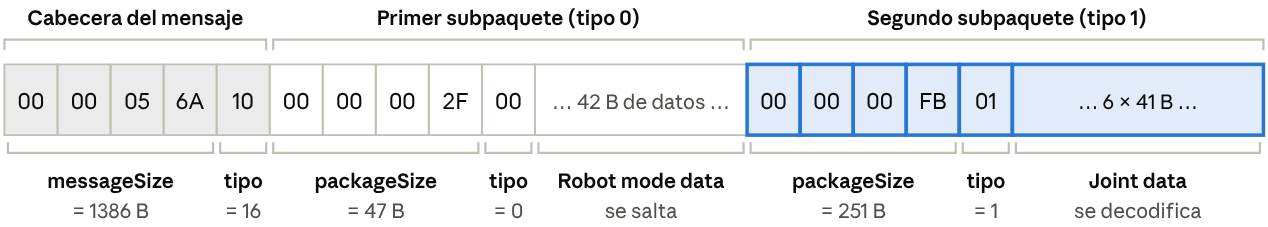
El bloque de una articulación es `>ddddddB`: 3 × 8 + 4 × 4 + 1 = **41 bytes**. El subpaquete *Joint data* mide 5 + 6 × 41 = **251 bytes**. Comprobadlo con `struct.calcsize` y verificad en ejecución que el `packageSize` recibido vale eso: si no, algo está desalineado.

> Si olvidáis el `>`, Python usará el orden nativo de vuestro PC (*little-endian*) y obtendréis ángulos del orden de 10^300 o cercanos a 0. Es el error más frecuente de la práctica.

Ejemplo: la cabecera de un mensaje, byte a byte

Los primeros bytes de un mensaje de estado tienen esta forma (los valores de longitud son ilustrativos: dependen de la versión de software):

Los primeros bytes de un mensaje de estado, campo a campo



Bytes en hexadecimal, big-endian. Longitudes ilustrativas: dependen de la versión de software.

Cabecera de un mensaje de estado · byte a byte

- `00 00 05 6A` en big-endian es 0x056A = 1386: el mensaje completo mide 1386 bytes, contando estos 4.
- `10` es 16: mensaje de estado del robot.
- El primer subpaquete mide 0x2F = 47 bytes y es de tipo 0; se salta avanzando 47 bytes desde su inicio.
- El siguiente mide 0xFB = 251 bytes y es de tipo 1: los datos articulares que buscáis.

> El documento de Universal Robots tiene alguna errata (por ejemplo, describe el campo `toolMode` como "estado de la articulación" y la tabla de modos de herramienta usa nombres `JOINT_MODE_*`). No os fíeis ciegamente de él: validad siempre las longitudes con lo que recibís.

7. La interfaz de tiempo real (puerto 30003)

La interfaz de tiempo real envía siempre **el mismo paquete con los mismos campos en las mismas posiciones**, sin subpaquetes ni tipos: para leer `q_actual` basta saltar directamente a su desplazamiento. En e-Series lo hace a 500 Hz, 50 veces más rápido que la primaria.

Origen: esta interfaz no aparece en el documento *Primary/Secondary Client Interface*. Su estructura completa está en la hoja de cálculo *Client Interfaces* de Universal Robots (pestaña *RealTime*), disponible en el aula virtual. Consultadla para la versión de vuestro URSim.

Desplazamiento (bytes)	Campo	Tipo	Tamaño
0	*Message Size*: longitud total del paquete	`int`	4 B
4	*Time*: tiempo desde que arrancó el controlador	`double`	8 B
12	*q target*: posiciones objetivo de las 6 articulaciones	6 × `double`	48 B
60	*qd target*: velocidades objetivo	6 × `double`	48 B
108	*qdd target*: aceleraciones objetivo	6 × `double`	48 B
156	*I target*: corrientes objetivo	6 × `double`	48 B
204	*M target*: pares objetivo	6 × `double`	48 B
252	***q actual*: posiciones reales de las 6 articulaciones**	**6 × `double`**	**48 B**
300	*qd actual*: velocidades reales	6 × `double`	48 B
…	Corrientes, posición cartesiana, temperaturas, modos, E/S digitales…	Consultad la hoja *Client Interfaces*	

Observad las diferencias con la primaria:

- **No hay campo de tipo.** Solo existe un tipo de paquete, así que no hace falta distinguirlos.
- **Casi todo es `double`**, incluso valores que son enteros o bits (los modos o las entradas digitales viajan como `double`).
- **La longitud sigue estando al principio** y la necesitáis igualmente: TCP sigue sin respetar los límites de los paquetes, así que la estrategia de lectura del siguiente apartado es la misma.
- **La longitud total depende de la versión** de software, porque cada versión ha ido añadiendo campos al final. No la escribáis como constante en el código: leedla del propio paquete. Los campos de las primeras posiciones no cambian, y por eso los desplazamientos de la tabla son estables.

> **Frecuencia y rendimiento.** A 500 Hz llegan unos 500 paquetes de más de 1 kB por segundo. Si vuestro programa dibuja la simulación con cada paquete, se quedará atrás y el robot de la pantalla irá con retraso creciente. Separad la **recepción** (consumir todo lo que llega y guardar solo el último estado) del **dibujo** (actualizar la simulación a un ritmo razonable, por ejemplo 30 o 60 veces por segundo).

8. Estrategia de lectura del buffer

No se puede hacer `datos = s.recv(1500)` y suponer que `datos` es un mensaje: hay que acumular los bytes en un buffer y cortar los mensajes usando la longitud que cada uno lleva al principio.

Qué puede devolver `recv()`

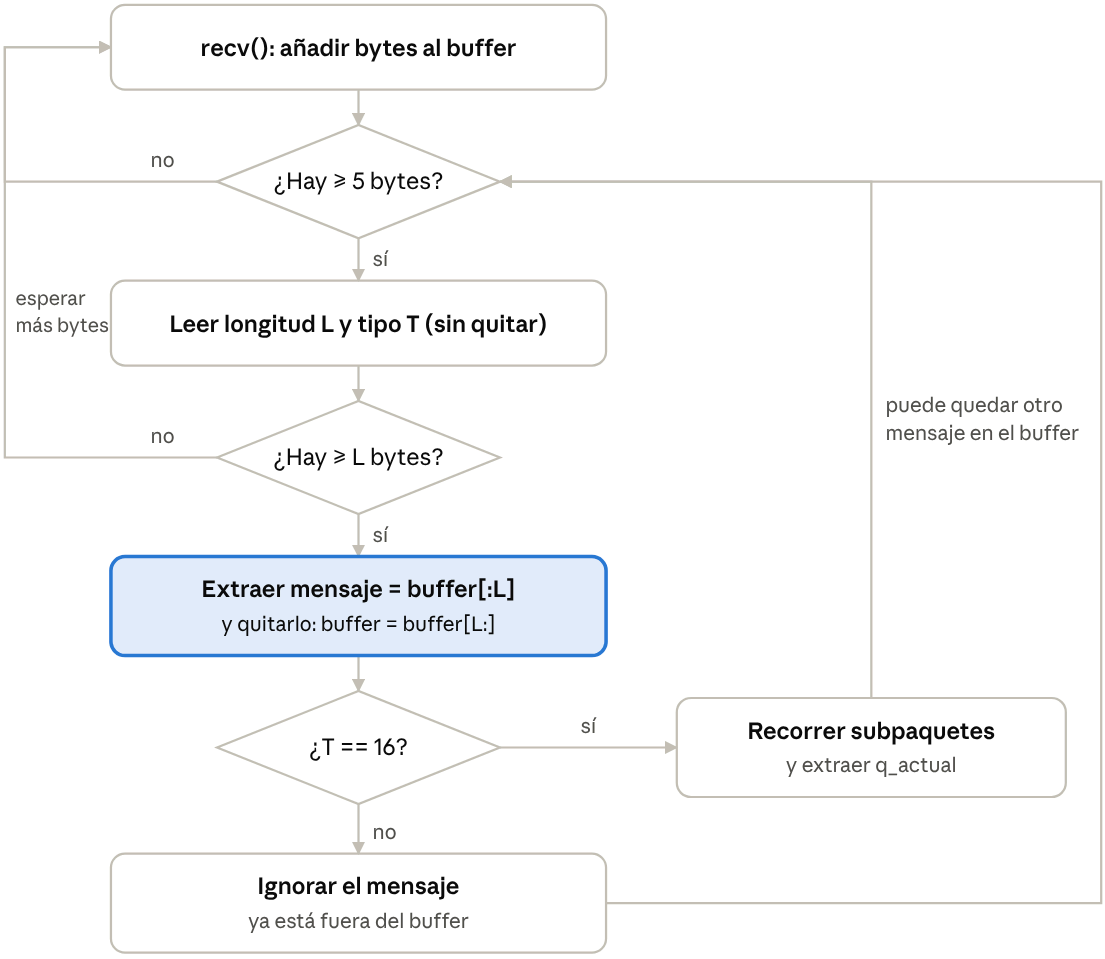
`recv(n)` devuelve **lo que haya llegado**, hasta `n` bytes. Según el momento en que lo llaméis, podéis encontraros cualquiera de estos casos:

Caso	Ejemplo	Qué pasa si lo tratáis como un mensaje completo
Mensaje incompleto	Llegan 600 bytes de un mensaje de 1386	`struct` falla o lee basura al final
Exactamente un mensaje	Llegan los 1386 bytes	Funciona… por casualidad
Más de un mensaje	Llega el final de uno y el principio del siguiente	Se pierde el segundo y el siguiente `recv()` empieza a mitad de mensaje: a partir de ahí todo está desalineado
Bytes antes de conectar	El primer mensaje es el de versión (tipo 20), no uno de estado	Si esperáis siempre el tipo 16, el primer mensaje rompe el programa

En una red local poco cargada el caso más frecuente es el segundo, y por eso un programa ingenuo parece funcionar en las pruebas y falla en el laboratorio cuando hay más tráfico o cuando usáis la interfaz de 500 Hz.

Algoritmo

Cortar el flujo TCP en mensajes usando la longitud de cada uno



Algoritmo de lectura del buffer · válido para 30001 y 30003

1. Llamad a `recv()` y añadid lo recibido al final de un `bytearray`. Si `recv()` devuelve 0 bytes, el robot ha cerrado la conexión: salid del bucle.
2. Mientras el buffer tenga al menos 5 bytes, leed **sin quitarlos** la longitud `L` (4 bytes) y el tipo `T` (1 byte).
3. Si el buffer tiene menos de `L` bytes, el mensaje no ha llegado entero: volved a `recv()`.
4. Si tiene al menos `L`, cortad los primeros `L` bytes (el mensaje) y eliminadlos del buffer.
5. Si `T` es 16, recorred sus subpaquetes. Si no, ignoradlo. En ambos casos volved al paso 2, porque puede haber otro mensaje completo esperando.

Recorrer los subpaquetes de un mensaje de estado

Dentro del mensaje ya cortado, la misma idea se repite a pequeña escala:

```
posicion = 5                                    # saltar la cabecera del
mensaje
mientras posicion < L:
    tam  = int de 4 bytes en posicion
    tipo = byte en posicion + 4
    si tipo == 1:
        para cada articulación j de 0 a 5:
            inicio_art = posicion + 5 + 41 * j
            q[j] = double en inicio_art
    posicion = posicion + tam                   # saltar al siguiente
subpaquete
```

Este pseudocódigo es orientativo: traducídlo vosotros a Python.

Validaciones que distinguen un programa robusto

- **Longitud imposible.** Si `L` es menor que 5 o desproporcionadamente grande (por ejemplo, más de 64 kB), el buffer está desalineado. Lo más sencillo y seguro es cerrar la conexión y volver a abrirla.
- **Subpaquete que se sale del mensaje.** Si `posicion + tam` supera `L`, el mensaje está corrupto o lo estáis interpretando mal: no sigáis leyendo.
- **Tamaño del subpaquete de articulaciones.** Si `tam` no vale 251, avisad: puede que la versión del robot tenga otro formato.
- **Cortes de conexión.** Si el robot se apaga o se reinicia URSim, el programa debe detectarlo (`recv` devuelve 0 o lanza una excepción) y reintentar la conexión sin bloquearse.

> **Para la interfaz de tiempo real el algoritmo es idéntico**, salvo que no hay campo de tipo: la cabecera mínima son 4 bytes y, una vez cortado el paquete, `q_actual` empieza en el byte 252.

9. El visualizador: paquete `UR3e_rg2`

El visualizador se proporciona como el paquete de Python `UR3e_rg2`. Su módulo `dynamic_ur3e` contiene la clase `UR3eRG2`, que abre una ventana de PyBullet con un UR3e, permite añadir más robots a la misma escena y colocar cualquiera de ellos en una configuración articular. "Teletransportar" el robot simulado consiste en una sola llamada, `move_robot()`, con los ángulos leídos del robot real.

Instalación

El paquete se distribuye como un *wheel* en el fichero `ur3e_rg2-0.1.0-py3-none-any.whl`, disponible en el aula virtual. Se instala con `pip`, que instala también sus dependencias (`pybullet`, `pybullet_planning` y `numpy`).

1. **Comprobad la versión de Python** con `python --version` en un terminal (PowerShell o símbolo del sistema en Windows, terminal en Linux). El paquete exige Python 3.9 o posterior.
2. **Cread un entorno virtual** en la carpeta de la práctica, para no mezclar estas librerías con las del sistema:

```
python -m venv venv
```

3. **Activad el entorno.** Al activarlo, `(venv)` aparece al principio de la línea del terminal. Hay que activarlo cada vez que abráis un terminal nuevo.

```
Windows:  venv\Scripts\activate
Linux:    source venv/bin/activate
```

4. **Instalad el wheel** desde la carpeta donde lo hayáis descargado:

```
pip install ur3e_rg2-0.1.0-py3-none-any.whl
```

En vuestros scripts importad siempre la clase desde ese módulo: `from UR3e_rg2.dynamic_ur3e import UR3eRG2`.

Métodos que vais a usar

Método	Qué hace	Cuándo lo usáis
`UR3eRG2()`	Abre la ventana y carga un primer UR3e en el origen, con identificador `0`	Una sola vez por programa: PyBullet admite una única ventana por proceso
`spawn_robot([x, y, z, roll, pitch, yaw])`	Añade otro UR3e con la base en esa posición (m) y orientación (rad) y devuelve su identificador	Notable: un robot simulado por cada robot real
`move_robot(id, q)`	Coloca de forma instantánea las articulaciones del robot `id` en los ángulos `q` (rad)	En cada actualización, con los 6 `q_actual` recibidos
`get_param_values()`	Devuelve un diccionario con el valor de los deslizadores de la ventana (claves `" j0"` a `" j11"`)	Sobresaliente: definir un punto con los deslizadores

Cómo pasar los ángulos a `move_robot()`

El modelo tiene 12 articulaciones móviles: las 6 del brazo (posiciones 0 a 5) y 6 adicionales que en esta práctica no se usan. Basta con pasar los 6 ángulos del brazo: `move_robot()` completa el resto con los valores actuales. Tened en cuenta tres detalles del módulo:

- **Pasad una lista, no una tupla.** `struct.unpack` devuelve una tupla y `move_robot()` falla con ella. Convertidla con `list()`.
- **La lista que pasáis se modifica.** El módulo le añade los 6 valores que faltan, así que después de la llamada tiene 12 elementos. Si vais a reutilizarla, pasad una copia: `move_robot(0, list(q))`.
- **Usad el identificador que devuelve `spawn_robot()`.** No es consecutivo al del primer robot, porque el suelo de la escena también ocupa un identificador.

Comprobad el convenio de ángulos

Antes de dar por buena la conexión, llevad URSim a una postura fácil de reconocer (por ejemplo, todas las articulaciones a 0° menos el hombro y la muñeca 1 a −90°) y comprobad que el robot simulado adopta exactamente la misma. Si alguna articulación aparece girada o con el signo cambiado, el modelo URDF usa un origen o un sentido distinto para esa articulación y debéis corregirlo en vuestro código, nunca en el módulo proporcionado.

Bucle principal

PyBullet no debe quedarse esperando a la red. Si el programa se queda bloqueado en `recv()`, la ventana deja de responder. Para el aprobado basta con un único bucle que reciba, decodifique y actualice. Para el notable necesitaréis separar la recepción de cada robot de la actualización de la ventana (siguiente apartado).

10. Desarrollo guiado por hitos

Avanzad por hitos pequeños y comprobables: no escribáis el programa entero y lo probéis al final. Cada hito tiene una comprobación que debéis superar antes de pasar al siguiente.

Bloque A · Interfaz primaria (script `primaria.py`)

Hito	Qué hacer	Cómo saber que está bien
A0	Configurar la VM en modo puente, arrancar URSim y comprobar `ping` y el puerto 30001	El puerto responde desde el PC
A1	Conectar al puerto 30001 e imprimir cuántos bytes devuelve cada `recv()`	Las cantidades varían: comprobad que no coinciden siempre con un mensaje
A2	Implementar el buffer e imprimir longitud y tipo de cada mensaje completo	Primero aparece un tipo 20 (versión) y después tipos 16 unas 10 veces por segundo, cada uno con la misma longitud
A3	Recorrer los subpaquetes de cada mensaje 16 e imprimir tipo y tamaño de cada uno	Las longitudes de los subpaquetes suman exactamente la longitud del mensaje menos 5
A4	Decodificar el subpaquete 1 e imprimir los 6 `q_actual` en grados	Coinciden con los valores de la pestaña *Mover* de PolyScope

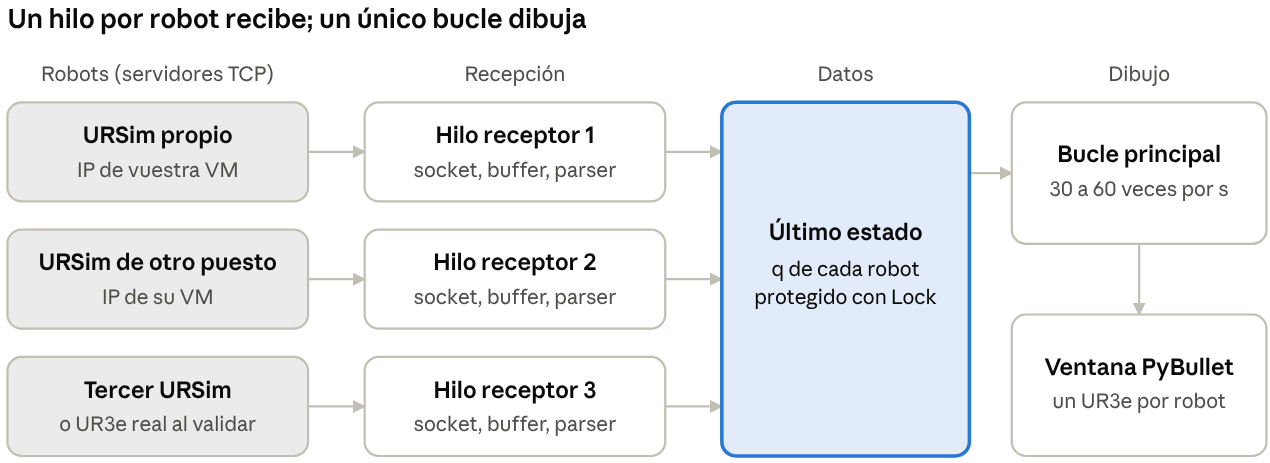
Hito	Qué hacer	Cómo saber que está bien
A5	Llamar a `move_robot()` (robot 0) con los ángulos recibidos	Al mover el robot en URSim, el de PyBullet le sigue

Bloque B · Interfaz de tiempo real (script `tiempo_real.py`)

Hito	Qué hacer	Cómo saber que está bien
B1	Conectar al puerto 30003 y aplicar el mismo buffer, ahora con cabecera de 4 bytes	Todos los paquetes tienen la misma longitud
B2	Contar paquetes por segundo	Unos 500 por segundo en e-Series
B3	Leer los 6 `q_actual` a partir del byte 252	Coinciden con los de `primaria.py`
B4	Actualizar PyBullet sin perder paquetes	Tras mover el robot rápido en URSim, el simulado no acumula retraso

Bloque C · Varios robots en el mismo programa

Con una sola conexión basta un bucle. Con varias, si el bucle se queda esperando en el `recv()` de un robot, los demás se congelan. La solución recomendada es un hilo receptor por conexión y un único bucle de dibujo:



Arquitectura recomendada para el notable · un hilo por conexión

- Cada **hilo receptor** tiene su propio socket y su propio buffer, y solo escribe el último `q` decodificado de su robot.
- El **bucle principal** lee ese estado a ritmo fijo y actualiza cada robot simulado. PyBullet se llama solo desde este bucle.
- El acceso al estado compartido se protege con un `threading.Lock` para no leer un vector a medio escribir.
- La lista de robots (IP, puerto, posición en la escena) se lee de un fichero de configuración o de los argumentos del programa, no se escribe en el código. Se crea un único `UR3eRG2` y un robot simulado por cada robot real con `spawn_robot()`.
- Si un robot no responde o se desconecta, los demás siguen funcionando y su robot simulado se marca de alguna forma (por ejemplo, deja de moverse y se avisa por consola).

Bloque D · Enviar órdenes

Para mover el robot no se envían tramas binarias: se envía **texto en URScript**, el lenguaje de programación del controlador de Universal Robots. El controlador recibe el texto por TCP, lo compila y lo ejecuta como un programa.

Por qué puerto

Aceptan URScript las interfaces 30001, 30002 y 30003; las de solo lectura (30011 y 30012) no. Una conexión TCP es bidireccional, así que podéis escribir las órdenes en el mismo socket del puerto 30001 del que estáis leyendo el estado. Es la opción recomendada. Si abrís una segunda conexión solo para enviar, tenéis que seguir leyendo y descartando lo que el robot envía por ella: si nadie lo lee, se llena el buffer de recepción y la conexión acaba bloqueada o cerrada.

Formato de las órdenes

Texto codificado en UTF-8 (en la práctica, ASCII) y **terminado en salto de línea** (`\n`). Sin el salto de línea el controlador sigue esperando y no ejecuta nada. Se envía con `sendall()`, que garantiza que se escriben todos los bytes.

- **Una orden suelta** cabe en una línea y se ejecuta como un programa de una sola instrucción:

```
movej([0.0, -1.5708, 0.0, -1.5708, 0.0, 0.0], a=0.5, v=0.3)
```

- **Varias órdenes encadenadas** deben ir dentro de una función `def … end`. El controlador ejecuta el programa completo. Si enviáis varias líneas sueltas, cada una sustituye a la anterior y solo se ejecuta la última.

```
def secuencia():
  movej([0.0, -1.5708, 0.0, -1.5708, 0.0, 0.0], a=0.5, v=0.3)
  movej([0.5, -1.2, 1.0, -1.4, -1.5708, 0.0], a=0.5, v=0.3)
  movej([-0.5, -1.2, 1.0, -1.4, -1.5708, 0.0], a=0.5, v=0.3)
end
```

Instrucciones que necesitáis

La referencia completa está en el *URScript Programming Language Manual* de Universal Robots.

Instrucción	Qué hace	Unidades
`movej(q, a, v)`	Movimiento articular hasta la configuración `q`, una lista de 6 ángulos	rad, rad/s², rad/s
`movel(p[x, y, z, rx, ry, rz], a, v)`	Movimiento lineal de la herramienta hasta una pose cartesiana	m y rad; m/s², m/s
`stopj(a)`	Detiene el movimiento con la deceleración `a`	rad/s²
`sleep(t)`	Espera `t` segundos dentro de un programa	s

Cómo saber que la orden se ha cumplido

El robot no responde a una orden: el único acuse de recibo es el ACK de TCP, que solo indica que los bytes han llegado, no que se hayan ejecutado. Hay que deducirlo del estado que el robot sigue publicando:

Secuencia de envío y confirmación de una orden

1. Tras enviar, esperad a que `isProgramRunning` (subpaquete 0) pase a verdadero. Si no ocurre en 1 o 2 segundos, la orden ha sido rechazada: buscad un mensaje de tipo 20 que lo explique.

2. Mientras sea verdadero, el robot está ejecutando. No enviíéis otra orden: **un programa nuevo aborta el que está en marcha**.
3. Cuando vuelva a falso, comprobad que `q_actual` está cerca del objetivo (por ejemplo, menos de 0,01 rad en cada articulación). Solo entonces la orden se considera cumplida.

Condiciones previas en el robot

El robot debe estar encendido, con los frenos liberados y en modo de seguridad *Normal* (`robotMode` = 7, *RUNNING*). En PolyScope 5, además, los programas que llegan por red solo se ejecutan en **modo remoto**: activad *Ajustes* → *Sistema* → *Control remoto* y seleccionad *Remoto* en la esquina superior derecha. En *Ajustes* → *Seguridad* → *Servicios* comprobad que la interfaz que usáis está habilitada y no en solo lectura. Los nombres exactos de los menús pueden variar según la versión de URSim.

> **Error peligroso:** enviar ángulos en grados. `movej` interpreta los valores como radianes, así que 90 se convierte en más de 14 vueltas. Validad antes de enviar que cada ángulo está entre −2π y 2π y que procede de un punto enseñado o de los deslizadores, nunca de un valor escrito a mano sin comprobar.

11. Entrega y evaluación

La evaluación es **incremental y binaria**: cada nivel se supera entero o no se supera, y para optar a un nivel hay que haber superado todos los anteriores. No hay notas intermedias dentro de un nivel.

Nivel	Nota	Requisitos (todos obligatorios)	Qué se comprueba en la defensa
Aprobado	5	Dos scripts separados: `primaria.py` (puerto 30001) y `tiempo_real.py` (puerto 30003). Cada uno obtiene los 6 `q_actual` de un robot y mueve con ellos el UR3e de PyBullet. Ambos usan buffer y delimitan los mensajes por su longitud	Se mueve el robot en URSim y el simulado le sigue con los dos scripts. Se reinicia URSim y el programa no se cuelga

Nivel	Nota	Requisitos (todos obligatorios)	Qué se comprueba en la defensa
Notable	7	Todo lo anterior, más un único script que visualiza **varios robots a la vez** (al menos dos) en la misma ventana, cada uno en su posición de la escena. Los robots se configuran sin tocar el código. Un robot que se desconecta no detiene a los demás	Se mueven dos robots a la vez (vuestro URSim y otro) y después se desconecta uno
Sobresaliente	10	Todo lo anterior, más el **envío de órdenes** y la mini-aplicación *Enseñar y reproducir* descrita abajo, funcionando contra URSim y validada en un UR3e real con el profesor	Se enseña una secuencia de al menos 4 puntos, se previsualiza y se ejecuta. Se provoca un error y el programa lo muestra y se detiene

Mini-aplicación del sobresaliente: *Enseñar y reproducir*

Es el ciclo básico de cualquier célula robotizada de manipulación: enseñar una tarea, comprobarla antes de ejecutarla y supervisar su ejecución. El visualizador sirve para **ver qué va a hacer el robot antes de que lo haga**.

1. **Enseñar.** El operador lleva el robot a una posición (en URSim desde la pestaña *Mover*; en el robot real en modo de guiado manual) y pulsa una tecla para guardar la configuración actual, leída de la interfaz primaria, como punto de la secuencia. También se puede definir un punto con los deslizadores de PyBullet.
2. **Ejecutar paso a paso.** El programa envía un `movej` por punto y **no envía el siguiente hasta comprobar con el estado del robot** que el anterior ha terminado (`isProgramRunning` falso y `q_actual` cercano al objetivo).
3. **Supervisar.** Si llega un mensaje de error (tipo 20, subtipos 6 o 10) o se activa una parada de protección (`isProtectiveStopped`), el programa detiene la secuencia, muestra el motivo y espera a que el operador la reanude o la cancele.
4. **Guardar.** La secuencia se puede guardar en un fichero y cargar en otra sesión.

Límites obligatorios en el robot real: velocidad articular máxima 0,3 rad/s y aceleración 0,5 rad/s² en las órdenes que enviéis, secuencia probada antes en URSim y profesor presente con la seta de emergencia al alcance.

Qué se entrega

- **Código:** los scripts de cada nivel alcanzado y un `README` con cómo ejecutarlos y configurar las IP de los robots.
- **Memoria breve** (máximo 6 páginas) con: una breve explicación del código desarrollado, con la estructura y estrategias de lectura utilizadas. La respuesta a las preguntas del apartado 12.
- **Defensa en el laboratorio:** demostración de los requisitos del nivel al que optáis. Cualquier miembro del grupo debe poder explicar cualquier línea del código entregado; si no puede, el nivel no se considera superado. La defensa se hace primero en el aula contra URSim y después en el laboratorio de robótica con un UR3e real.

12. Preguntas de comprensión

Responded en la memoria. Las preguntas marcadas con (N) o (S) solo son obligatorias si optáis al notable o al sobresaliente.

1. Registrad durante un minuto cuántos bytes devuelve cada llamada a recv(). ¿Coincide siempre con la longitud de un mensaje? ¿Qué consecuencia tiene eso para vuestro código?
2. ¿Por qué el mensaje y cada subpaquete empiezan con su longitud? ¿Qué podría hacer vuestro programa con un subpaquete de un tipo que no aparece en la documentación?
3. Si decodificáis `q_actual` sin indicar *big-endian*, ¿qué valores obtenéis? Explicad por qué con un ejemplo de 8 bytes.
4. Comparad la interfaz primaria y la de tiempo real: ¿qué ventaja tiene cada una? ¿Cuál elegiríais para un visualizador que alimenta un panel de supervisión? ¿Y para registrar la trayectoria de un movimiento rápido?
5. ¿Por qué en NAT no podéis conectaros a URSim desde el PC sin configurar nada más, y en modo puente sí? ¿Qué equipo de la red ve la dirección MAC de vuestra VM?
6. Desconectad el cable de red del PC durante 5 segundos y volved a conectarlo. ¿Qué hace vuestro programa? ¿Qué haría TCP con los datos durante ese tiempo?
7. (N) Medid el retraso entre el movimiento de URSim y el del robot simulado con la interfaz primaria y con la de tiempo real. ¿De qué depende?
8. (N) ¿Qué ocurriría si el bucle de PyBullet leyera el estado compartido sin `Lock` mientras un hilo lo está escribiendo?
9. (S) ¿Por qué no basta con haber enviado un `movej` para saber que el robot ha llegado al punto? ¿Qué información del propio robot usáis para confirmarlo?

10. (S) Cualquier equipo de la red puede conectarse al puerto 30001 y enviar un programa al robot. ¿Qué riesgo supone en una planta real? Proponed dos medidas, una en la configuración del robot y otra en la arquitectura de red.